

ML

Lecture 1-2

Data Collection $\rightarrow$ Feature Design $\rightarrow$ Model Training $\rightarrow$ Model Validation.

Feature Extraction. What feature is can be used as discriminative... say Cats from dogs.

create mathematical model, train & then validate

Features are also called COVARIATES

Lecture 1-3

$Y = f(X) + \epsilon$; $f \rightarrow$ Systemic Information, ϵ is random error

ML: How to learn f ?

Prediction

x_i are available but

y_i are not known.

we estimate $\hat{y}_i$ or $\hat{y}$ by learning $\hat{f}$.

$$E(Y - \hat{y})^2 = E[f(x) + \epsilon - \hat{f}(x)]^2$$

$$= [f(x) - \hat{f}(x)]^2 + \text{Var}(\epsilon)$$

We want to estimate f & reduce the reducible error ϵ

Inference

Both x_i & y_i are known.

Try to understand how x_i 's affect y_i 's

y

Categorical or Nominal

Continuous

Categorical with order or ordinal (Ex Grades)

Problem Class

— Classification

— Regression

— Ordinal Regression

Estimate f

Define Training Sample $S = (x_1, y_1), \dots, (x_n, y_n)$, where $x_i = (x_{i,1}, x_{i,2}, \dots, x_{i,p})^T$

Each $x_i \in \mathbb{R}^p$

There are n subjects

$\Rightarrow X \in \mathbb{R}^{p \times n}$

$$x_i = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{ip} \end{pmatrix}$$

n = # Samples (or data points) or Subjects

p = # features

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & & \vdots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

Tasks

Classification - Category, Label; Text Classification, Pattern Recognition, Speech Recognition

L-1-4

Regression

- Learning: Learning a real value: Stock Values.

Clustering

- Learn to partition a set into homogenous subsets: Communities within larger grp of ppl.

Dimension Reduction

- Learning transformation from original representation to into a lower dimensional representation: Digital Image Processing

Terminology (Contd.)

Examples - Instances of data for learning or evaluations

Features - Attributes representing an ex.

Labels - CATEGORIES OR VALUES assigned to examples.

Hyperparam - Free Parameters; Not learned during training; Free inputs

Training Sample - examples used in the training process.

Test Sample - examples used for validation

Loss Function - F'n measuring the diff of predicted label & true label. We minimize this f'n

Hypothesis Set - Possible functions which map the feature vectors to a set of labels

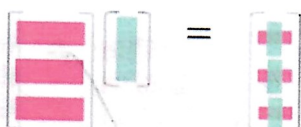
Learning Scenarios

Online Learning: Intermixed Training & Testing. Data comes over time
minimize cumulative Loss over all rounds.

Reinforcement Learning: Intermixed Training & Testing. Learner interacts with and affects environment. Rewards & penalties. Exploration - Try new strategy
Exploitation - Use the learned Knowledge model

Self-Supervised Learning: Do not have a lot of label label
Learner interactively collects training examples
used when labels are expensive or difficult to obtain.

Mv1



The row vectors of A are multiplied by a vector x and become the three dot-product elements of Ax .

$$Ax = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} (x_1 + 2x_2) \\ (3x_1 + 4x_2) \\ (5x_1 + 6x_2) \end{bmatrix}$$

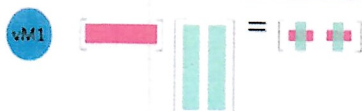
Mv2



The product Ax is a linear combination of the column vectors of A .

$$Ax = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1 \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix} + x_2 \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

Once used to viewing it as (Mv2), Ax is easily seen as a linear combi of the columns of A . Those products fill the column space of A denoted as $C(A)$. The Soln space of $Ax=0$ is the nullspace of A denoted as $N(A)$



$$yA = [y_1 \ y_2 \ y_3] \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = [(y_1 + 3y_2 + 5y_3) \ (2y_1 + 4y_2 + 6y_3)]$$

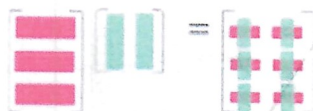


$$yA = [y_1 \ y_2 \ y_3] \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = y_1 [1 \ 2] + y_2 [3 \ 4] + y_3 [5 \ 6]$$

A row vector y is multiplied by the two column vectors of A and become the two dot-product elements of yA .

The product yA is a linear combination of the row vectors of A

MM 1



Every element becomes a dot product of row vector and column vector

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} = \begin{bmatrix} (x_1 + 2x_2) & (y_1 + 2y_2) \\ (3x_1 + 4x_2) & (3y_1 + 4y_2) \\ (5x_1 + 6x_2) & (5y_1 + 6y_2) \end{bmatrix}$$

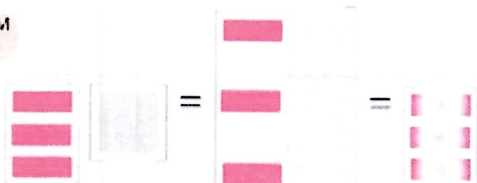
MM 2



Ax and Ay are linear combinations of columns of A .

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} = A \begin{bmatrix} x & y \end{bmatrix} = [Ax \ Ay]$$

MM 3



The produced rows are linear combinations of rows.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} = \begin{bmatrix} a_1' \\ a_2' \\ a_3' \end{bmatrix} x = \begin{bmatrix} a_1' x \\ a_2' x \\ a_3' x \end{bmatrix}$$

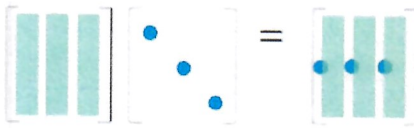
MM 4



Multiplication AB is broken down to a sum of rank 1 matrices.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_1 & a_2 \end{bmatrix} \begin{bmatrix} b_1' \\ b_2' \end{bmatrix} = a_1 b_1' + a_2 b_2' \\ = \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \end{bmatrix} + \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix} \begin{bmatrix} b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} b_{11} & b_{12} \\ 3b_{11} & 3b_{12} \\ 5b_{11} & 5b_{12} \end{bmatrix} + \begin{bmatrix} 2b_{21} & 2b_{22} \\ 4b_{21} & 4b_{22} \\ 6b_{21} & 6b_{22} \end{bmatrix}$$

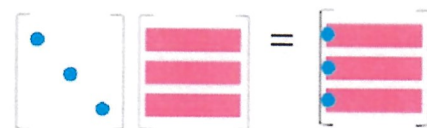
P1'



Applying a diagonal matrix from the right scales each column.

$$AD = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} d_1 & & \\ & d_2 & \\ & & d_3 \end{bmatrix} = \begin{bmatrix} d_1 a_1 & d_2 a_2 & d_3 a_3 \end{bmatrix}$$

P2'



Applying a diagonal matrix from the left scales each row.

$$DB = \begin{bmatrix} d_1 & & \\ & d_2 & \\ & & d_3 \end{bmatrix} \begin{bmatrix} b_1^* \\ b_2^* \\ b_3^* \end{bmatrix} = \begin{bmatrix} d_1 b_1^* \\ d_2 b_2^* \\ d_3 b_3^* \end{bmatrix}$$

P3



This pattern makes another combination of columns.
You will encounter this in differential/recurrence equations.

$$XDc = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} d_1 & & \\ & d_2 & \\ & & d_3 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = c_1 d_1 x_1 + c_2 d_2 x_2 + c_3 d_3 x_3$$

P4



A matrix is broken down to a sum of rank 1 matrices,
as in singular value/eigenvalue decomposition.

$$U\Sigma V^T = \begin{bmatrix} u_1 & u_2 & u_3 \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \sigma_3 \end{bmatrix} \begin{bmatrix} v_1^T \\ v_2^T \\ v_3^T \end{bmatrix} = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \sigma_3 u_3 v_3^T$$

L-2-2

Linear Regression Model

$$\hat{y} = \text{model}(\vec{x}, \vec{w}^*) = \vec{x}_0^* \cdot \vec{w}_0^* + \vec{x}_1^* \vec{w}_1^* + \vec{x}_2^* \vec{w}_2^* + \dots + \vec{x}_N^* \vec{w}_N^* \\ = \vec{w}_0^* + \vec{x}_1^* \vec{w}_1^* + \vec{x}_2^* \vec{w}_2^* + \dots + \vec{x}_N^* \vec{w}_N^*$$

 $\vec{x}^* = \text{Input}$ $\vec{w}^* = \text{Optimal set of weights found by minimizing a regression cost function.}$ Measures of Accuracy

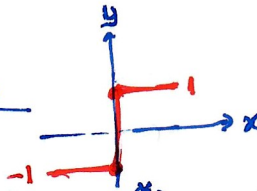
1) MSE (Least Squares) : $\text{MSE} = \frac{1}{N} \sum_{i=1}^n (\text{model}(\vec{x}_i, \vec{w}^*) - y_i)^2$ - Sensitive to outliers

2) Mean Absolute Deviation (MAD) or Least Absolute Deviation : $\text{MAD} = \frac{1}{N} \sum_{i=1}^n |\text{model}(\vec{x}_i, \vec{w}^*) - y_i|$
- Better if the forecast DOES NOT have asymmetric distribution

Using Linear Regression Model for ClassificationDEFINITION- Labels: $y_p = \{-1, +1\}$

$$\text{sign}(y) = \begin{cases} +1 & \text{if } y \geq 0 \\ -1 & \text{if } y < 0 \end{cases}$$

$$\text{model}(\vec{x}, \vec{w}^*) = \text{sign}(x^T \vec{w}^*) \\ = \text{sign}(\vec{w}_0^* + \vec{x}_1^* \vec{w}_1^* + \dots + \vec{x}_N^* \vec{w}_N^*)$$

Let $y_i = \text{true label}$, $\hat{y}_i = \text{Predicted label}$ & $\hat{y}_i = \text{model}(\vec{x}_i, \vec{w}^*)$

Define an indicator function

$$I[\hat{y}_i = y_i] = \begin{cases} 0 & \text{if } \hat{y}_i \neq y_i \\ 1 & \text{if } \hat{y}_i = y_i \end{cases}$$

• # Correctly labeled samples = $\sum_{i=1}^N I[\hat{y}_i = y_i]$ - (1)

• Accuracy = $\frac{1}{N} \sum_{i=1}^N I[\hat{y}_i = y_i]$ - (2) Range of Accuracy = $[0, 1]$

- Accuracy is easy

- But could be misleading for highly imbalanced data

• Say we have 95% dog pictures & 5% cat pictures. Let's say the model classifies all of them as dogs, then accuracy = 95% BUT it missed ALL CATS

Balanced Accuracy

$$A_{\text{balanced}} = \frac{(A_{+1}) + (A_{-1})}{2},$$

where $A_{+1} = \frac{1}{|\Omega_{+1}|} \sum_{i \in \Omega_{+1}} I[\hat{y}_i = y_i]$ &

$$A_{-1} = \frac{1}{|\Omega_{-1}|} \sum_{i \in \Omega_{-1}} I[\hat{y}_i = y_i]$$

 Ω dimension of the set Ω_i $A_{+1} = \text{Accuracy for correctly labeled } +1 \text{ samples}$ $A_{-1} = \text{Accuracy for correctly labeled } -1 \text{ samples}$ $\Omega_{+1} = \text{Set of samples classified as } +1$ & $\Omega_{-1} = \text{Set of samples classified as } -1$

Confusion Matrix

$$\text{TPR (Sensitivity): } A_+ = \frac{TP}{TP + FN}$$

$$\text{TNR (Specificity): } A_- = \frac{TN}{TN + FP}$$

$$\text{FPR: } 1 - \text{Specificity} = \frac{FP}{TN + FP}$$

$$\text{Acc.: } \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Bal. Acc.: } A_{\text{Bal}} = \frac{1}{2} A_+ + \frac{1}{2} A_- = \frac{1}{2} \frac{TP}{TP + FN} + \frac{1}{2} \frac{TN}{TN + FP}$$

	Total Population	Predicted	
		Positive	Negative
Actual	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

$$\text{Precision: } \frac{TP}{TP + FP}$$

$$\text{F}_1\text{-Score: } \frac{2TP}{2TP + FP + FN}$$

$$\text{Recall: } \frac{TP}{TP + FN}$$

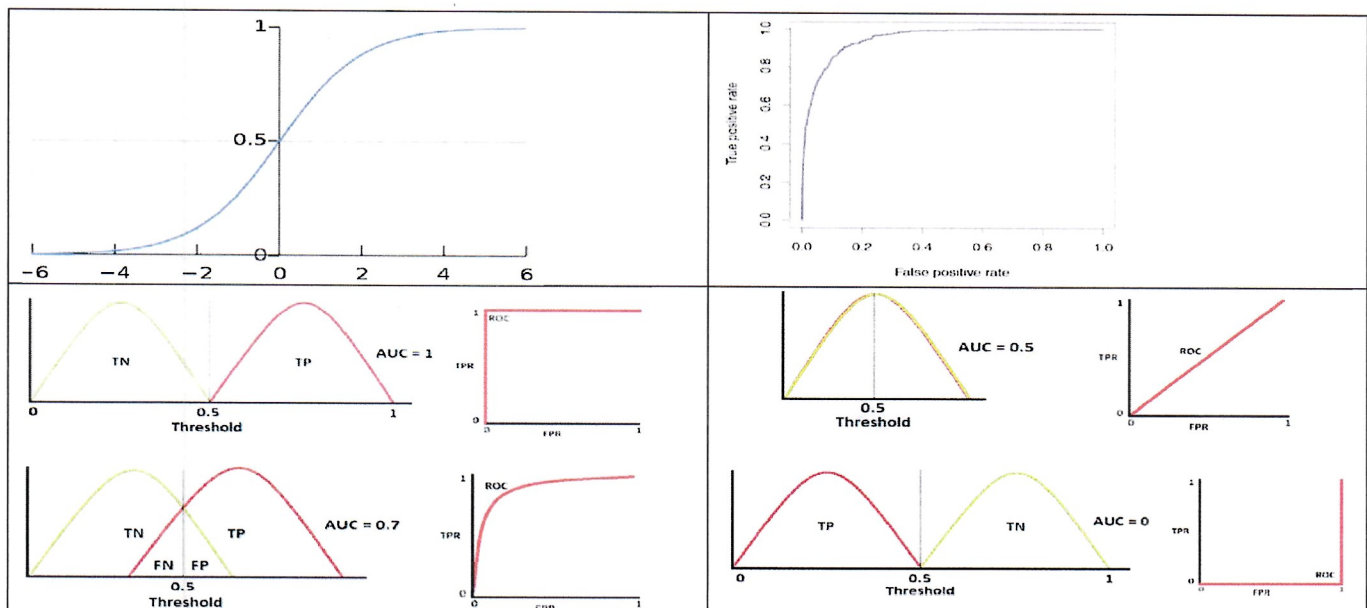
$$\frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

ROC (Receiver Operator Curve)

- A probability curve used in **Binary Classification Problems**
- By predicting probability instead of label, can generate ROC Curve

AUC-ROC Curve

- Measure of separability - Performance measurement for at various threshold settings
- It tells how much a model is capable of distinguishing between classes
- an choose threshold on ROC curve that optimizes some threshold-specific measurement.
- Can also plot Precision-Recall Curve



KNN (K-Nearest Neighbors)Notations

length of vector $x = \langle x_1, x_2, \dots, x_D \rangle = \sqrt{x_1^2 + x_2^2 + \dots + x_D^2} = \|\langle x_1, x_2, \dots, x_D \rangle\|_2 = \|\langle x_1, x_2, \dots, x_D \rangle\|_2^2 = \|\langle x_1, x_2, \dots, x_D \rangle\|_2^2$ Assumed

OR $\|\langle x_1, x_2, \dots, x_D \rangle\|_2^2 = \|\langle x_1, x_2, \dots, x_D \rangle\|_2^2 = x_1^2 + x_2^2 + \dots + x_D^2$ Note $\|\langle x_1, x_2, \dots, x_D \rangle\|_2$

Training Data N samples

$$S^{\text{train}} = \{(x_1, y_1), \dots, (x_N, y_N)\}$$

Used for learning $f: x \rightarrow y$ Testing Data M samples

$$S^{\text{test}} = \{(x_1, y_1), \dots, (x_M, y_M)\}$$

Used to evaluate how well $f(\cdot)$ does in predicting an unseen sampleNote $S^{\text{train}} \cap S^{\text{test}} = \emptyset$ (Must be so)Set NotationsLet $S = \{1, 2, 3, 8, 25\}$, then $S \setminus \{8\} = S - \{8\} = \{1, 2, 3, 25\}$ KNN - Classify data into one of the classes

- Let's say we have C classes, then each $y \in \{1, 2, 3, \dots, C\}$
- Let the input data be D dimensional i.e. $\vec{x} = \{x_1, x_2, \dots, x_D\}$
- Let the model be $f: x \rightarrow y$

Special Case
 $C=2$, then
 then
 $y \in \{0, 1\}$ or
 $y \in \{-1, +1\}$ or
 $y \in \{1, 2\}$

 $K=1$ Let $\vec{x}'$ be a sample from Test Set

$$\text{Let } nn(x') = \arg \min_{i=1, \dots, N} \|\vec{x}' - \vec{x}_i\|_2^2 = \arg \min_{i=1, \dots, N} \left(\sum_{j=1}^D (x'_j - x_{ij})^2 \right)$$

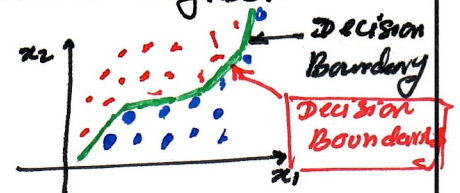
Model: $\hat{y} = f(x) = y_{nn(x)}$

$$nn_1(x) = \arg \min_{i \in \{1, \dots, N\}} \|\vec{x}' - \vec{x}_i\|_2^2$$

Returns the index i for the nearest neighbor. $K=2$

$$nn_2(x) = \arg \min_{i \in \{1, \dots, M\} \setminus \{nn_1(x')\}} \|\vec{x}' - \vec{x}_i\|_2^2$$

i would now exclude the nearest neighbor above.
 We are finding the second nearest neighbor now



$$\underline{K=3} \quad nn_3 = \arg \min_{i \in \{1, \dots, M\} \setminus \{nn_1(x'), nn_2(x')\}} \|\vec{x}' - \vec{x}_i\|_2^2$$

We are now finding the third nearest neighbor so we exclude from i the first & second nearest neighbors

Set of K nearest neighbors

$$Knn(x') = nn_K(x) = \arg \min_{i \in \{1, \dots, N\} \setminus \{nn_1(x'), \dots, nn_{K-1}(x')\}} \|\vec{x}' - \vec{x}_i\|_2^2$$

Set of K -nearest neighbors = $Knn(x') = \{nn_1(x'), \dots, nn_K(x')\}$

KNN Model Recall $knn(x') = \{m_{n_1}(x'), m_{n_2}(x'), \dots, m_{n_k}(x')\}$

- Every member in the set $knn(x')$ votes for in favor of its own class.
- Count the votes for each class

$$V_c = \sum_{K \in knn(x')} I[y_K = c], \quad c = 1, 2, \dots, C$$

- Assign x' to the class with most votes.

$$\hat{y} = f(x') = \arg \max_{c=1, \dots, C} V_c$$

Value of K does not depend on data x, X .

K is our first hyperparameter.

If $K \downarrow$: many cluttered small regions with complex Decision Boundary

If $K \uparrow$: Fewer larger regions. Simpler Decision Boundary.
More errors